
EC NB-IoT 低功耗开发手册

EigenCOMM Wireless Microcontroller

文档说明

本文档详细描述移芯通信 NB-IoT 芯片的低功耗能力，指导用户通过使用相关 API 完成低功耗应用的开发。

目录

1. 引言	4
1.1 文档目的	4
1.2 内容简介	4
1.3 相关名词	4
1.3.1 睡眠深度	4
1.3.2 投票	4
1.3.3 回调函数	4
1.3.4 DeepSleep Timer	4
1.3.5 Retention IO	4
1.3.6 User NVMem	4
1.3.7 MCU Mode	4
2. EC616/616s 硬件资源	5
2.1 PMU 硬件框图	5
3. EC616/616s 低功耗软件	5
3.1 睡眠流程	5
3.1.1 Idle 流程	5
3.1.2 Sleep1 流程	6
3.1.3 Sleep2 流程	6
3.1.4 Hibernate 流程	7
3.2 睡眠深度控制	7
3.2.1 应用层睡眠深度确定依据	8
3.2.2 应用层睡眠深度控制	8
3.3 睡眠前后回调函数	10
3.4 DeepSleep Timer	12
3.5 User NVMem	13
3.6 其他 API	14
3.6.1 AON IO 配置	14
3.6.2 AON IO 使用流程	14
3.6.3 获取上次睡眠状态	15
3.6.4 获取上次唤醒源	15
3.6.5 MISC	16
4. 应用低功耗实现	17
4.1.1 应用低功耗开机处理流程	17
4.1.2 应用上下文恢复相关 API	18
4.1.3 应用低功耗恢复流程	18
5. 双芯片方案的唤醒	19
6. 测试	25
6.1 唤醒中断响应测试	25
6.2 双芯片唤醒方案二测试	28
7. 错误码	30

8. 注意事项及常见问题	30
8.1 API 使用限制及解决方法	30
9. 版本	32
10. 关于我们	33

1. 引言

1.1 文档目的

本文档详细描述移芯通信 NB-IoT 芯片 EC616/616s 的低功耗能力，指导用户通过使用相关 API 完成低功耗应用的开发。

1.2 内容简介

本文简要介绍了 EC616/616s 低功耗软硬件架构，讲解低功耗相关 API 的使用方法，使用注意事项。并通过一个例子向用户展示如何在 EC616/616s 上开始低功耗软件的开发。

1.3 相关名词

1.3.1 睡眠深度

ACTIVE 状态：不开启低功耗模式，即使无事可做，MCU 依然处于循环等待状态，功耗较大。

IDLE 状态：MCU 在无任务时会关闭核心工作时钟，任何中断都能唤醒系统，并重启核心时钟。

SLEEP1 状态：在 IDLE 基础上对所有外设掉电，外设中断无法唤醒系统。

SLEEP2 状态：在 SLEEP1 基础上关闭 256KB Memory，仅保留 16KB Retention Memory。

HIBERNATE 状态：在 SLEEP2 基础上，关闭 16KB Retention Memory。

1.3.2 投票

EC616/616s 是一个多任务系统，多任务平等的决定睡眠深度。最终睡眠的深度受多种因素影响，最终 EC616/616s 会尽可能进入一个能进入的最深睡眠模式。

1.3.3 回调函数

由用户编写的函数，并注册到 SDK 中，在 SDK 执行过程中的某个阶段自动调用用户注册的函数。

1.3.4 DeepSleep Timer

支持在深睡眠模式下（SLEEP2 及 HIBERNATE）运行的 timer。

1.3.5 Retention IO

支持在深睡眠模式下（SLEEP2 及 HIBERNATE）保持电平状态的 IO

1.3.6 User NVMem

提供用户一段 SRam 用于存储掉电会丢失的信息，数据实际最终写入 Flash，但 SDK 会确保尽量少的写入。只在确实能进入 SLEEP2/HIBERNATE 的情况下才会真正做写入操作

1.3.7 MCU Mode

一种特殊的工作状态，系统主频为 16MHz，此情况下 PLL 关闭，功耗较低

2. EC616/616s 硬件资源

2.1 PMU 硬件框图

EC616/616s 拥有 256KB+16KB Sram，20 个 Normal IO（BGA 封装）或 10 个 Normal IO（QFN）。EC616 拥有 6 个 Retention IO（BGA 封装），EC616s 无 Retention IO。高于 SLEEP1（含）的睡眠深度，仅支持 RTC Wakeup 和 Pad Wakeup，IDLE 状态下任何外设中断均可唤醒系统。对于 EC616s，较 EC616 多了一套低功耗 UART，可直接从串口唤醒系统，并且串口数据不会丢失。各睡眠状态下，模块的带电状态如下表：

表一：

	ACTIVE	IDLE	SLEEP1	SLEEP2	HIBERNATE
256KB Sram	ON	ON	ON	OFF	OFF
16KB Ret Sram	ON	ON	ON	ON	OFF
Normal IO	ON	ON	OFF	OFF	OFF
Ret IO	ON	ON	ON/OFF	ON/OFF	ON/OFF

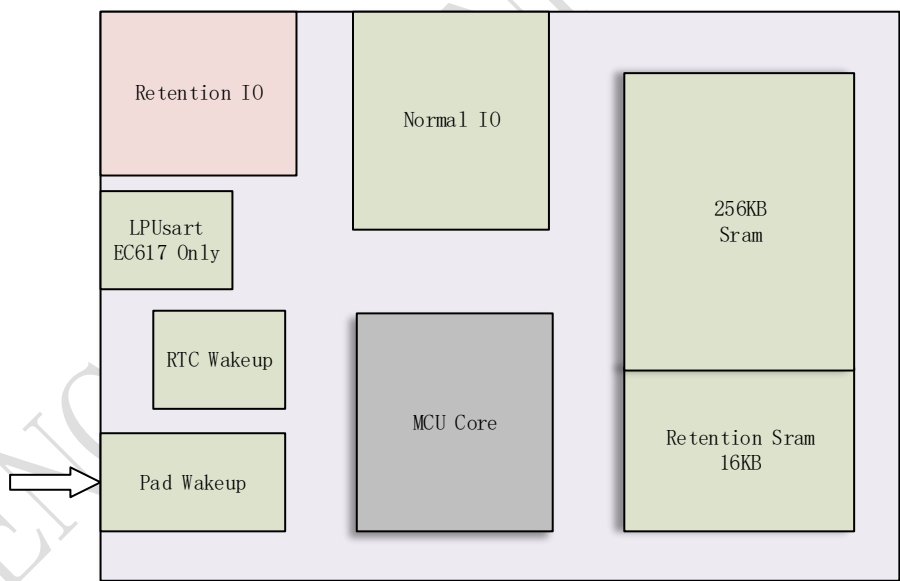


图 1: EC616/616s 低功耗硬件框图

3. EC616/616s 低功耗软件

3.1 睡眠流程

3.1.1 Idle 流程

程序在空闲时软件自动尝试进入 idle 状态，从 idle 状态唤醒后，程序（PC 值）从原地开始执行。

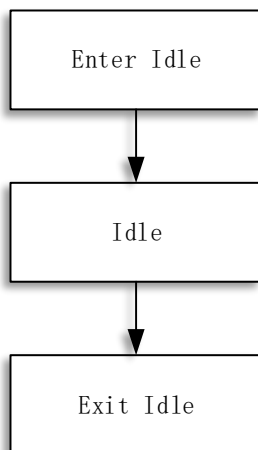


图 2: IDLE 流程

3.1.2 Sleep1 流程

Sleep1 状态下，由于外设会掉电。因此在真正进入 Sleep1 前需要执行一段 PreSleep 函数，唤醒后同样执行一段 PostSleep 函数来恢复睡眠前的状态。在此，用户可以使用 API（将在之后章节介绍）自行注册回调函数，用户注册的回调函数应该尽可能的短小，不得使用使系统挂起的操作（如收发操作系统消息，等待任务队列，调用操作系统延时 OsDelay 等）、耗时操作(如擦写 flash，等待外部中断，等待串口数据等)。为保证较低的功耗及睡眠流程的正常运行，原则上，用户注册的所有 PreSleep 回调函数，所有 PostSleep 回调函数，均不得超过 3ms。驱动级别的寄存器保存及恢复由 SDK 负责完成，无需用户单独考虑。

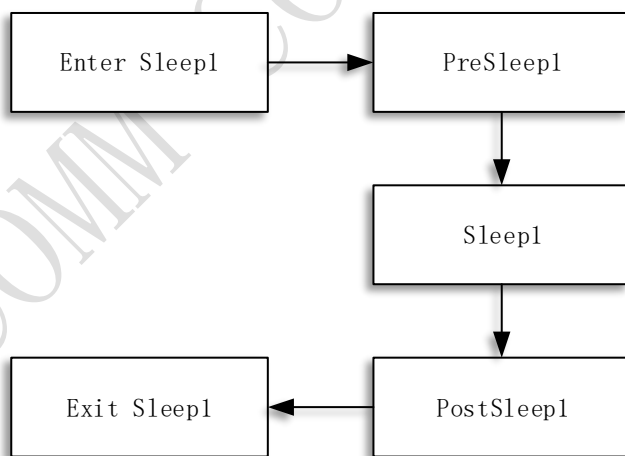


图 3: SLEEP1 流程

3.1.3 Sleep2 流程

Sleep2 状态下，256KB Sram 会掉电，仅保持 16KB Retention Sram 有电。假如睡眠成功，系统唤醒后程序将从头运行。在完成一段 NB 流程后，用户将在 Main_Entry 函数拿到控制权。此时用户可通过 API 函数（将在之后章节介绍），查询状态获得此次是程序第一次上电运行还是从睡眠流程中唤醒。假如睡眠失败，程序将顺序执行下去。

在真正进入睡眠前，无法获知睡眠将成功或失败。因此，用户需要做好失败准备，即注册相应的回调函数到 PostSleep2 中。同样的，PreSleep2 和 PostSleep2 的函数时间需要严格控制，以尽可能的发挥低功耗优势。

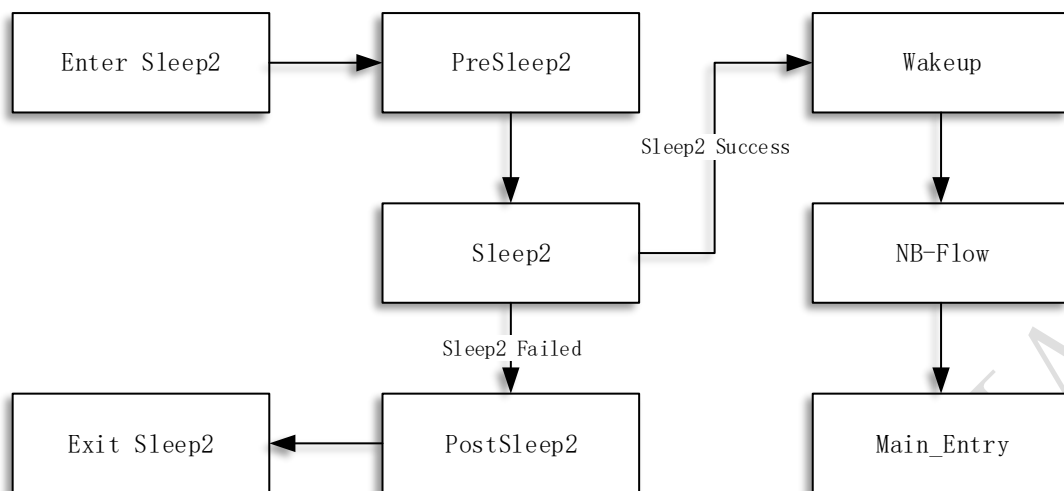


图 4: SLEEP2 流程

3.1.4 Hibernate 流程

Hibernate 状态下，16KB Sram 也将掉电。流程上同 Sleep2 基本一致。同样的，用户也可以注册 PostHib 回调函数，为 Hibernate 失败做好准备。

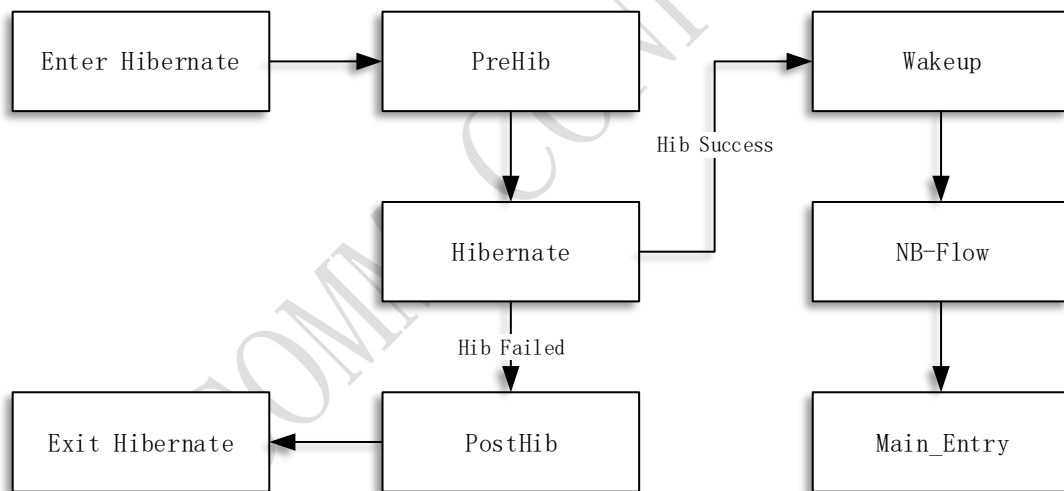


图 5: HIBERNATE 流程

3.2 睡眠深度控制

EC616/616s 具有四级睡眠深度控制：IDLE，SLEEP1，SLEEP2，HIBERNATE

为尽可能的实现低功耗，用户应当选择可进入的最深睡眠模式。AT 模式下睡眠深度设置方法如下：

AT+ECPMUCFG=1,4 将最大睡眠深度设置到 Hibernate。

AT+ECPMUCFG=1,3 将最大睡眠深度设置到 Sleep2。

AT+ECPMUCFG=1,2 将最大睡眠深度设置到 Sleep1。

AT+ECPMUCFG=1,1 将最大睡眠深度设置到 Idle。

AT+ECPMUCFG=0 不使能低功耗模式。

3.2.1 应用层睡眠深度确定依据

对于应用层程序开发人员，确定最深睡眠深度的依据主要有，Sram 需求及外设使用情况。用户需要知道的原则是 SLEEP1 及其之后深度的睡眠模式外设掉电。SLEEP2 仅有 16KB Sram（其中部分用户无法使用），HIBERNATE 状态下无可用 Sram 空间。举例如下：

表二：

	IDLE	SLEEP1	SLEEP2	HIBERNATE
等待 Usart 中断	YES	NO	NO	NO
收发 SPI 数据中	YES	NO	NO	NO
间歇收发 Usart 数据	YES	YES	YES/NO	YES/NO
程序需要 200KB SRam	YES	YES	NO	NO
程序需要 3KB Sram	YES	YES	YES	NO

3.2.2 应用层睡眠深度控制

EC616/616s 作为一款低功耗 NB-MCU，睡眠深度的确定是依赖多种因素的。用户可以通过 API 函数投票决定进入某种级别的睡眠深度，最终 EC616/616s 进入哪一种睡眠深度还依赖 NB 底层程序。用户通过 API 的投票决定了应用层允许的最大睡眠深度，SDK 会综合考虑并确保下一次睡眠深度不高于用户指定的深度。睡眠投票的机制如下图：

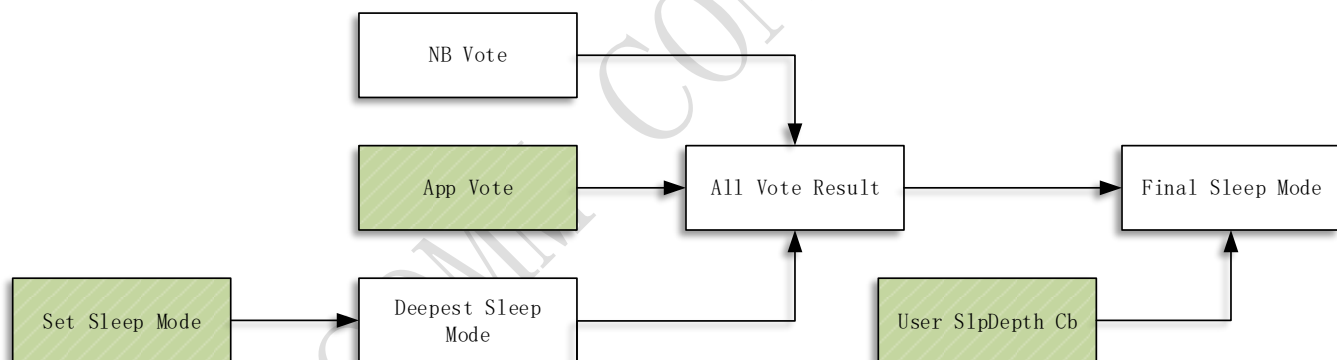


图 6：投票决策树

如上图所示绿色部分为开放给用户进行睡眠深度控制的控制点，对应不同的 API 函数。

App Vote: 用户通过申请投票句柄（Handle），并对句柄投票来控制睡眠深度

Set Sleep Mode: 直接控制最大睡眠深度。在某些应用场景中，无需做到最低功耗，即可将最大睡眠深度调低。另外，在调试期间也可通过控制最大睡眠深度，方便测试。

User SlpDepth Cb: 注册用户的睡眠深度回调函数，此函数在真正进入睡眠流程前调用。例如，在双芯片方案中，主控在对 EC616/616s 某硬件拉高电平后，可控制 NB 不进入休眠模式。

以上三个控制点相应的 API 函数如下，在此仅简单介绍函数名称及使用场合，具体说明文档请见 doxygen 或 slpman_ec616/616s.h。

（1）App Vote API 函数

```
slpManRet_t slpManApplyPlatVoteHandle(const char* name, uint8_t *handle)
```

申请一个投票句柄，并记录句柄名称。句柄名称会保存（最大长度为 9 个 byte），之后可根据句柄名称查询到句柄。

用户应当保证句柄名称的唯一性。可根据函数返回值获知函数运行情况，返回值及错误码见附录。

`slpManRet_t slpManGivebackPlatVoteHandle(uint8_t handle)`

向系统返还一个投票句柄。返还后改句柄所有信息将被从系统清除。用户应当在句柄销毁前清除对该句柄的投票，即如果对某句柄调用了 1 次 `slpManPlatVoteDisableSleep` 函数，在销毁前要保证 `slpManPlatVoteEnableSleep` 已经被调用过了，保证投票的配对性。

`slpManRet_t slpManFindPlatVoteHandle(const char* name, uint8_t *handle)`

可根据名称重新获取对应的句柄。在申请句柄时请确保名称的唯一性

`slpManRet_t slpManPlatVoteDisableSleep(uint8_t handle, slpManSlpState_t status)`

对某个句柄，取消某深度及以上的睡眠。例如：

`slpManPlatVoteDisableSleep(handle1, SLP_SLP2_STATE);` //禁止 Sleep2，即只能进入 IDLE，SLEEP1

`slpManPlatVoteDisableSleep(handle2, SLP_HIB_STATE);` //禁止 HIBERNATE，即只能进入 IDLE，SLEEP1，SLEEP2

`slpManRet_t slpManPlatVoteEnableSleep(uint8_t handle, slpManSlpState_t status)`

对某个句柄，使能某深度的睡眠。例如：

`slpManPlatVoteEnableSleep (handle1, SLP_SLP2_STATE);` //取消之前对 Sleep2 睡眠深度的禁止操作

`slpManPlatVoteEnableSleep (handle2, SLP_HIB_STATE);` //取消之前对 HIBERNATE 睡眠深度的禁止操作

特别注意：

1. 申请句柄后，对同一句柄只能投票同一种睡眠深度。例如，只能 `slpManPlatVoteDisableSleep(handle1, SLP_SLP2_STATE)`，或 `slpManPlatVoteEnableSleep(handle1, SLP_SLP2_STATE)`。
不可在执行 `slpManPlatVoteDisableSleep(handle1, SLP_SLP2_STATE)`后，再执行 `slpManPlatVoteEnableSleep(handle1, SLP_HIB_STATE)`
2. 对句柄的投票可在多线程中调用，并引入了投票计数。在 n 次 `slpManPlatVoteDisableSleep` 函数后，调用 n 次 `slpManPlatVoteEnableSleep` 函数才能真正将投票清除。
3. **强烈建议**检查返回值，确保每次操作执行成功，即返回值为 `RET_TRUE`。返回类型 `slpManRet_t` 对应的枚举类型可以在 `slpman_ec616/616s.h` 中找到。假如未检查返回值导致以为投票成功了，实际投票失败，出现问题后将很难定位。

补充介绍，如何理解函数名：

1. Disable SLP_SLP2_STATE 理解为在 SLEEP2 状态前加一道锁，PMU 在尝试睡眠途中遇到 SLEEP2 这个锁，就无法进入更深的睡眠状态
2. Enable SLP_SLP2_STATE 理解为取消 SLEEP2 状态的一道锁，之后是否能进入 HIBERNATE 还要看是否在 HIBERNATE 加了锁，假如没有锁，就能成功进入
3. Disable SLP_SLP2_STATE 两次理解为对 SLEEP2 加了两把锁。因此只有 Enable SLP_SLP2_STATE 两次，即摘除两把锁才能真正把 SLEEP2 上的锁清理干净
4. 假如未对 SLP2 加锁的情况下再去 Enable SLP_SLP2_STATE，结果是没有可以摘除的锁，返回 `RET_VOTE_SLP_UNDERFLOW`。这种情况下给出的错误只是给用户一个警告，对于使能 SLP2 的睡眠是没有问题的。

(2) Set Sleep Mode API 函数

```
void slpManSetPmuSleepMode(bool pmu_enable, slpManSlpState_t mode, bool save2flash)
```

设置 PMU 使能，最大睡眠深度。并设置是否将此配置写入 flash 中。系统在进入 hibernate 或 sleep2 并唤醒后系统使用的变量会丢失，假如应用层不想再配置一次，那么在第一次设置睡眠深度时可以选择将此配置写入 flash。SDK 会负责在系统初始化过程中读出配置并设置生效。为尽可能的减少 flash 写入的次数，在 open-mcu 模式下，尽可能的不保存设置，因为系统唤醒后可以很容易的再次调用函数并设置 PMU 使能及控制睡眠深度。

(3) User SlpDepth Cb API 函数

```
void slpManRegisterUsrSlpDepthCb(pmuUserdefSleepCb_Func callback)
```

此处设置的回调函数会在最终进入睡眠前被调用，回调函数内不能有耗时的操作、挂起系统或等待消息。对于大部分的睡眠控制使用投票决定，但对于不方便参与投票的情况可使用注册回调的方法实现睡眠控制。例如：在某系统中，NB 的某引脚为高时，保持系统唤醒状态。当此引脚为低电平时，才进入睡眠。此时可注册回调函数，函数中读取引脚电平，并根据电平高低确定睡眠深度。

3.3 睡眠前后回调函数

进入不同深度睡眠后，部分 sram 掉电，部分外设掉电。因此睡眠前后需要有休眠前准备流程（PreSleep）和唤醒后恢复流程（PostSleep）。此流程通过回调注册 API 注册到系统中，如前所述，此处 API 必须为不耗时操作，不挂起系统，不等待信号量。原则上，用户注册的所有回调函数执行时间不应超过 3ms。

回调函数分为两类：Predefined 和 Userdefined。其中 Predefined 类型的回调函数在回调列表中占用固定的位置，编译后即确定位置。Userdefined 类型回调函数的位置由注册的前后顺序确定。因此建议将对调用顺序敏感的操作放在 Predefined 类型中，对调用顺序不敏感的放在 Userdefined 类型中。目前，外设驱动均使用 Predefined 类型，驱动部分的睡眠恢复在移芯 SDK 已经实现，用户无需自己编写。参见 slpman_ec616/616s.h 中枚举类型：slpCbModule_t，该枚举类型定义如下：

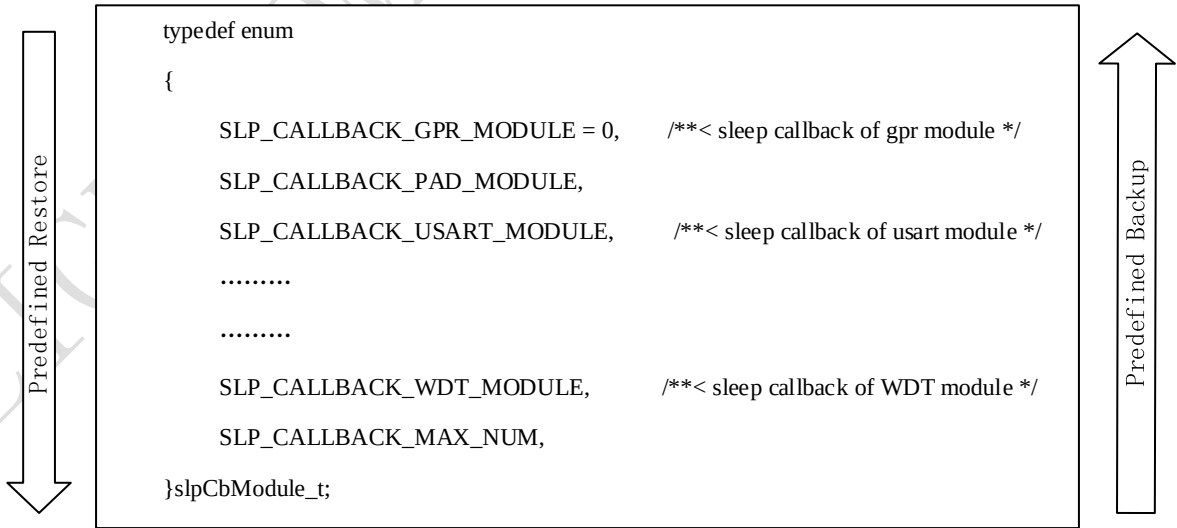


图 7：回调执行

PreSleep 流程执行从下至上，即先保存 WDT 模块，最后保存 GPR 模块。

PostSleep 流程执行从上至下，即先回复 GPR 模块，最后恢复 WDT 模块。

对于 Userdefined 类型的回调函数，由于使用程序动态注册，注册先后顺序依照程序运行先后顺序来确定。用户编写的回调函数推荐放于 Userdefined 类型中。对于 Userdefined 类型，PreSleep 及 PostSleep 流程中将依照注册先后顺序依次调用。

API 函数如下，在此仅简单介绍函数名称及使用场合，具体说明文档请见 doxygen 或 slpman_ec616/616s.h。

(1) Predefined callback

slpManRet_t slpManRegisterPredefinedBackupCb(slpCbModule_t module, slpManBackupCb_t backup_cb, void *pdata, slpManLpState state)
注册 Predefined 类型的 PreSleep 回调函数。其中，state 参数确定该回调函数在何种睡眠模式下被调用；pdata 参数为回调函数调用时的入参。
slpManRet_t slpManRegisterPredefinedRestoreCb(slpCbModule_t module, slpManRestoreCb_t restore_cb, void *pdata, slpManLpState state)
注册 Predefined 类型的 PostSleep 回调函数。其中，state 参数确定该回调函数在何种睡眠模式下被调用；pdata 参数为回调函数调用时的入参。
slpManRet_t slpManUnregisterPredefinedBackupCb(slpCbModule_t module)
注销某模块的 PreSleep 回调函数
slpManRet_t slpManUnregisterPredefinedRestoreCb(slpCbModule_t module)
注销某模块的 PostSleep 回调函数

(2) Userdefined callback

slpManRet_t slpManRegisterUsrdefinedBackupCb(slpManBackupCb_t backup_cb, void *pdata, slpManLpState state)
注册 Usrdefined 类型的 PreSleep 回调函数。其中，state 参数确定该回调函数在何种睡眠模式下被调用；pdata 参数为回调函数调用时的入参。
slpManRet_t slpManRegisterUsrdefinedRestoreCb(slpManRestoreCb_t restore_cb, void *pdata, slpManLpState state)
注册 Usrdefined 类型的 PostSleep 回调函数。其中，state 参数确定该回调函数在何种睡眠模式下被调用；pdata 参数为回调函数调用时的入参。
slpManRet_t slpManUnregisterUsrdefinedBackupCb(slpManBackupCb_t backup_cb)
注销某模块的 PreSleep 回调函数
slpManRet_t slpManUnregisterUsrdefinedRestoreCb(slpManRestoreCb_t restore_cb)
注销某模块的 PostSleep 回调函数

(3) 回调执行

回调函数在注册后由 SDK 负责调用和执行，用户无需调用回调执行函数。

uint8_t slpManExcutePredefinedBackupCb(slpManLpState state)
执行 Predefined 类型的 PreSleep 回调函数

uint8_t slpManExcutePredefinedRestoreCb(slpManLpState state)
执行 Predefined 类型的 PostSleep 回调函数
uint8_t slpManExcuteUsrdefinedBackupCb(slpManLpState state)
执行 Userdefined 类型的 PostSleep 回调函数
uint8_t slpManExcuteUsrdefinedRestoreCb(slpManLpState state)
执行 Userdefined 类型的 PostSleep 回调函数

3.4 DeepSleep Timer

除 SLEEP1 及 IDLE 状态不存在 sram 掉电的情况，其他几种状态（SLEEP2 及 HIBERNATE）在睡眠成功后均会导致 RTOS 的 OsTimer 丢失。因此，如果要维持所有 OsTimer 的运行，需要控制系统不能进入大于 SLEEP2 的睡眠深度。SDK 提供 8 个 DeepSleep Timer，在进入 SLEEP2 或 HIBERNATE 状态后可依然维持运行。

DeepSleep Timer API 如下：

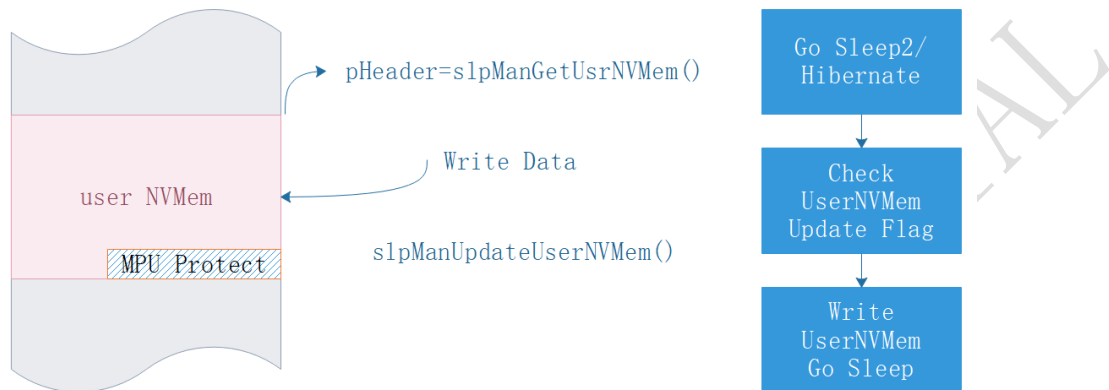
void slpManDeepSlpTimerRegisterExpCb(slpManUsrDeepSlpTimerCb_Func cb)
注册 DeepSleep Timer 的超时回调函数。SDK 会在 DeepSleep Timer 超时后调用此回调函数。所有 User DeepSleep Timer 使用同一回调函数，可以从回调函数的参数获得 DeepSleep Timer 的 ID。建议在 BSP_CustomInit 中注册超时回调，如在用户任务中注册回调，会因为超时的時候回调还未注册，而无法进入回调函数。
typedef void (*slpManUsrDeepSlpTimerCb_Func)(slpManTimerID_e id);
User DeepSleep Timer 回调函数类型。回调函数入参为 timer id，表示超时发生的 timer id。
void slpManDeepSlpTimerStart(slpManTimerID_e timerId, uint32_t nMs)
启动一个 DeepSleep Timer。设置 timerID 及超时时间，超时时间最大值为 580 小时，单位为 1ms。假如创建一个大于 580 小时的 DeepSleep Timer，该 Timer 会在 580 小时超时。timerId= DEEPSLP_TIMER_ID7 时，超时最大值为 2 小时。
bool slpManDeepSlpTimerIsRunning(slpManTimerID_e timerId)
查询某 timerId 的 DeepSleep Timer 是否在运行中。
void slpManDeepSlpTimerDel(slpManTimerID_e timerId)
删除某 timerId 的 DeepSleep Timer

注意：对于 DeepSleep Timer 不提供 Timer 暂定的功能。假如要暂停 Timer，可以先调用 slpManDeepSlpTimerDel 删除 Timer，并在需要重启 Timer 的时候调用 slpManDeepSlpTimerStart 重启 Timer。建议控制 DeepSleep Timer 的使用频率，高频的使用会导致 flash 寿命降低，因为每次创建这个 timer 都会做一次 flash 写入。

某些应用中可能比较难于去降低 DeepSleep Timer 的使用频次，针对这种状况我们专门将 slpManTimerID_e 中的 DEEPSLP_TIMER_ID7 做了特殊处理，启动这个 timer 不会对 flash 有写入操作，但需要注意的是这个 timer 的计数范围只有 2 小时，且实际精度为 1s(调用 slpManDeepSlpTimerStart 启动时传入参数依然是 ms 单位，但内部计数最小精度为 1s)

3.5 User NVMem

考虑到用户在休眠前时常需要保存一些数据到 flash 中，来实现下次从深睡眠唤醒后软件流程的恢复。因此，专门为用户预留了一块 SRam，向此 SRam 区域中写入的数据将在进入深睡眠前由 SDK 写入 flash 中，并在唤醒后重新从 flash 恢复到 SRam。用户可以认为此区域中的数据是可以一直维持的，但需要注意的是尽可能少的去更新这块区域，以减少 flash 擦写的频率。实现原理如下：



- (1) User NVMem 位于 Sram 中的某一块空间，用户通过 slpManGetUsrNVMem 函数获得此空间的首地址。
- (2) 获取首地址后用户可直接操作此地址，可使用空间为 2016Byte。可直接指定指针、数组、结构体等到这块空间上。为防止访问越界，在此区域最后 32 Byte 做了一块 MPU 只读保护，因此实际可访问空间为 2016 Byte。
- (3) 用户对此空间写入数据后，调用 slpManUpdateUserNVMem 来更新写入标志。
- (4) 当 SDK 最终判断能进入 Sleep2 或 Hibernate 时，查询是否需要写入此用户数据区到 Flash 中。
- (5) 系统从深睡眠唤醒后，SDK 会在初始化阶段从 Flash 恢复此区域的数据。
- (6) 目前电信 CTWING 和 OC 平台均使用这块区域做数据备份。

相关 API 如下：

void slpManUpdateUserNVMem(void);
告知 SDK，用户数据区的内容刚刚发生一次更新了。
uint8_t * slpManGetUsrNVMem(void);
获取用户数据区的首地址指针，用户可以使用改地址后的（2048-32）byte 长度的空间。用户深睡眠需要保持的变量可以直接写入此区域。写入后务必调用 slpManUpdateUserNVMem 函数告知 SDK 此区域有更新，这样在深睡眠前才能出发写入。
void slpManFlushUsrNVMem(void);
立即将此用户数据区中的内容写入到 Flash 中，而不是等待睡眠前再更新到 Flash 中。建议用户尽量少的使用此功能，由 SDK 自动在深睡眠前写入会极大的减少 Flash 写入的频率。

3.6 其他 API

3.6.1 AON IO 配置

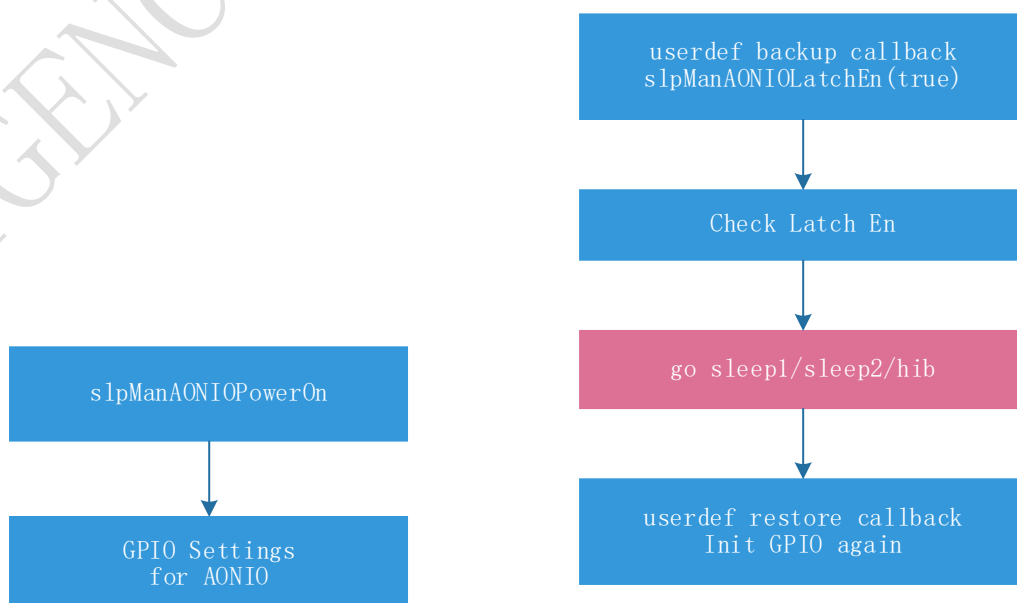
AON IO 仅仅在 EC616 上才有，EC616s 不存在 AONIO。

对于 BGA 封装的 EC616，拥有 6 个 Retention IO，可以在进入睡眠后保持 IO 电平为睡眠前的状态。这 6 个 GPIO 在 Active 的状态可以使用 GPIO 驱动来配置。但需要注意的是，需要对 AON IO 上电后才能进行配置，否则由于 AON IO 没电，GPIO 的配置将无效。相关配置 API 如下：

<code>void slpManAONIOLatchEn (IOLatchEn en)</code>
设置使能 AON IO。仅当使能后，Retention IO 才会在睡眠时保持电平状态，否则 Retention IO 掉电。Retention IO 电平的设置使用 GPIO 的 API 驱动来实现。Latch 仅在睡眠前一刻发生，如果调用此函数后 AON IO 电平发生变化，则已睡眠前一刻的电平值作为最终 Latch 的值
<code>IOLatchEn slpManAONIOGetLatchCfg (void)</code>
获取 AON IO 的使能状态，即读出 slpManAONIOLatchEn 的配置结果
<code>void slpManAONIOPowerOn(void)</code>
AON IO 默认状态下不带电，通过调用此函数对 AON IO 上电，上电后再使用 GPIO 驱动操作 AON IO。
<code>void slpManAONIOPowerOff (void)</code>
对所有 AON IO 下电，下电后所有 6 个 AON IO 都将掉电。同时睡眠后也不在 Latch 电平。
<code>void slpManAONIORelease(void)</code>
假如睡眠前配置了 Latch AON IO，系统唤醒后应该尽快配置 GPIO 并撤销 AON IO 的 Latch。用户在 Bsp_CustomInit 内应完成 Latch release。

3.6.2 AON IO 使用流程

AONIO 的配置及初始化流程如下左图：



AONIO 在睡眠前后的流程如上右图:

1. AONIO默认上电不使能, 假如需要使用需要, 应先将AONIO上电
 2. 随后AONIO的配置同普通GPIO完全一致
 3. AONIOLatchEn配置后GPIO并不是立即latch的, 而要等到真正进入sleep才会latch。从另一方面说, slpManAONIOLatchEn(true)调用之后仍然可以改变GPIO的状态。
 4. 也可以在睡眠前的回调函数中使能AONLatch
 5. 睡眠流程会在执行完回调后开始检查用户是否有latch需求
 6. 对于sleep1, 唤醒后会先执行UserdefRestoreCallback, 需要在此重新初始化GPIO, 仅需调用gpio配置即可, 无需重新AONIOPowerOn
 7. 在执行完UserdefRestoreCallback之后, SDK内部会执行AONIO Release操作, 因此需要确保在退出UserdefRestoreCallback之前已经将AONIO对应的GPIO配置到需要的电平
 8. 对于sleep2/hib, 睡眠失败也会导致AONIO的release。但GPIO配置并没有因为睡眠丢失, 因此无需在UserdefRestoreCallback重新配置GPIO
 9. 对于睡眠成功的情况, aonio会在BSP_CustomInit后release, 需要在此之前重新配置GPIO
 9. AONIO Release后引脚上的电平将会变成GPIO中配置的值
 10. AONIO Release会发生在
 - (1) BSP_CustomInit之后
 - (2) 睡眠失败后
 - (3) sleep1唤醒后
- 为在睡眠前后保持AONIO, 建议总结如下:
1. AONIO的配置应在BSP_CustomInit中完成, 将GPIO初始化为默认的状态。
 2. 注册 sleep1 状态的 UserdefRestoreCallback, 其中将 GPIO 配置为默认状态。

3.6.3 获取上次睡眠状态

EC616/616s 系统唤醒后, 可以通过 API 获取前一次睡眠状态。由于 SLEEP2 及 HIBERNATE 唤醒后系统将从头开始运行, 因此获取前一次睡眠状态变得尤为重要。在用户 main_entry 函数中针对 SLEEP2 和 HIBERNATE 的情况用户可以自定义不同的流程来实现不同的唤醒处理。通过如下 API 获取上一次的睡眠状态:

slpManSlpState_t slpManGetLastSlpState(void)

获取上次睡眠状态。

函数返回以下几种状态: SLP_SLP1_STATE, SLP_SLP2_STATE, SLP_HIB_STATE, SLP_ACTIVE_STATE

注意在程序流程上, SLEEP1 唤醒后由于程序将从睡眠前位置原地开始执行, SLEEP2 和 HIBERNATE 将重新开始运行 main_entry 函数。

3.6.4 获取上次唤醒源

通过 API 可以查询上次唤醒的唤醒源。唤醒源可以是 POR (表示本次其实是 reset 或上电后, 还未进入睡眠), RTC, PAD。

slpManWakeSrc_e slpManGetWakeupSrc(void)

获取上次唤醒源。

函数返回以下几种状态: WAKEUP_FROM_POR, WAKEUP_FROM_RTC, WAKEUP_FROM_PAD

3.6.5 MISC

除以上 API，EC616/616s PMU SDK 还提供一些投票状态查询函数。

bool slpManDrvSafeToSleep(void)
查询是否因有外设处于工作状态而无法进入睡眠。当有外设在工作的工作下，EC616/616s 只能进入 IDLE 状态。
void slpManGetDrvBitmap(uint32_t *bitmap)
获取外设驱动的投票状态，通过此接口可查看当前是因为哪个外设的工作导致系统无法睡眠。
void slpManGetPlatBitmap(uint32_t *slp_bitmap, uint32_t *slp2_bitmap, uint32_t *hib_bitmap)
获取平台层对 SLEEP1，SLEEP2 及 HIBERNATE 的投票结果。返回的结果是 bitmap，bit 为 0 表示可以进入当前级别的睡眠，bit 为 1 表示无法进入这一层次的睡眠。其中 bit 的位置与 slpManApplyPlatVoteHandle 申请到的 handle 一一对应。。例如查询到 slp_bitmap=0x02; 表示 handle 1 此时投票不能进入 SLEEP1。
slpManRet_t slpManFindPlatVoteHandle(const char* name, uint8_t *handle)
根据投票句柄 handle，查询对应的句柄名称。
slpManSlpState_t slpManPlatGetSlpState(void)
获取平台层投票结果。
slpManRet_t slpManDrvVoteSleep(slpDrvVoteModule_t module, slpManSlpState_t status)
外设驱动的投票，通常此函数 SDK 中的驱动程序会负责调用。用户一般无需关心。
void slpManSetWakeupPad(padWakeupSettings_t cfg)
配置 Wakeup 唤醒使能，可选择开启上升沿，下降沿，双沿唤醒 对于 EC616，6 个 Wakeup Pad 分 3 组。Wakeup0/1 为分组 0，Wakeup2/3 为分组 1，Wakeup4/5 为分组 2 const padWakeupSettings_t cfg = { false,true, // group0 posedge, negedge true, false, // group1 posedge, negedge false, false, // group2 posedge, negedge }; 如上述配置，将 Wakeup0/1 配置为下降沿唤醒，Wakeup2/3 配置为上升沿唤醒，Wakeup4/5 不使能外部唤醒。 建议尽量采用下降沿唤醒，保持常态为高电平。Wakeup Pad 上高电平无漏电流，低电平会出现 9uA 的漏电流。 需要特别注意，EC616 在唤醒状态下，无法识别到 Wakeup Pad 上到来的下降沿。

4. 应用低功耗实现

4.1.1 应用低功耗开机处理流程

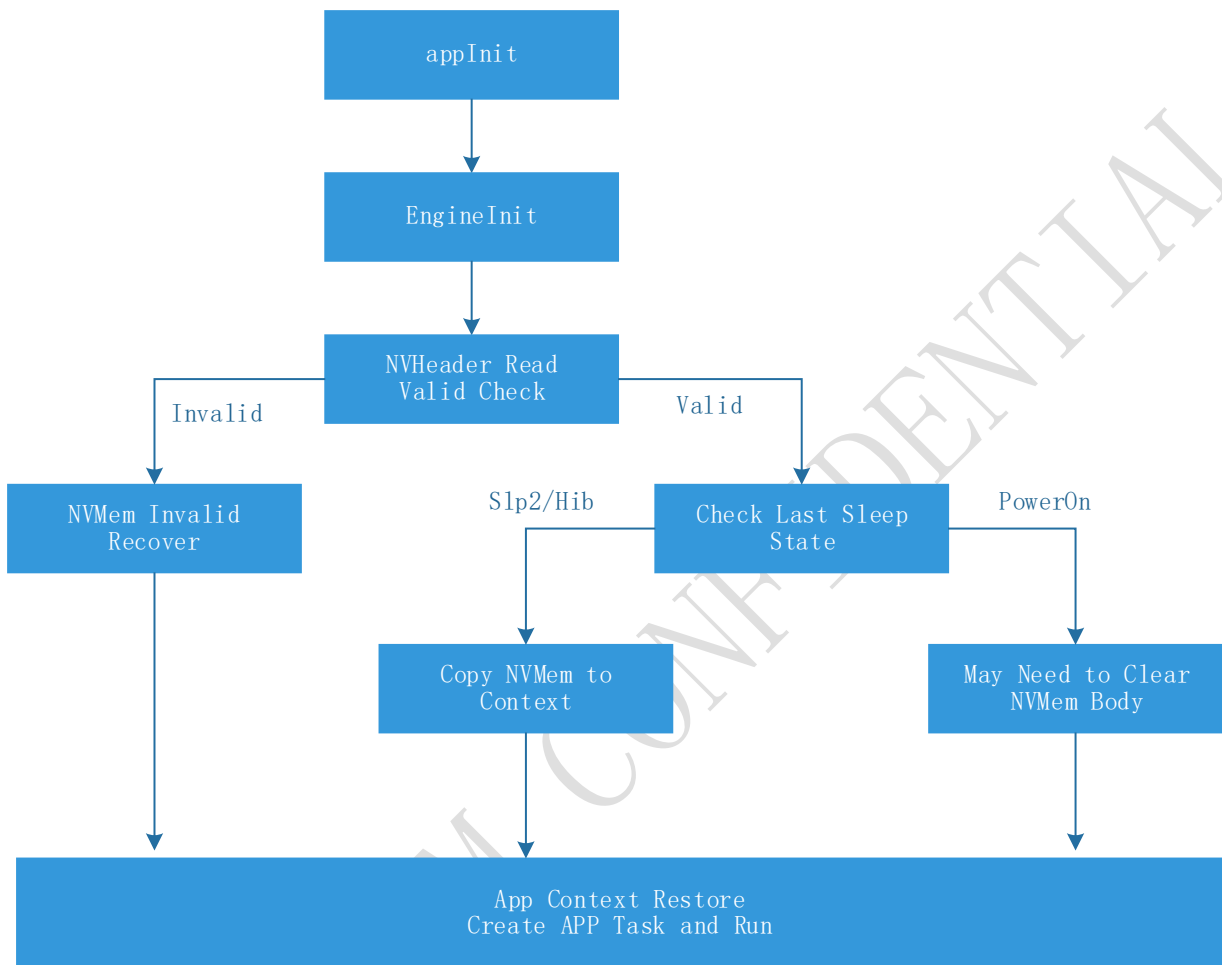
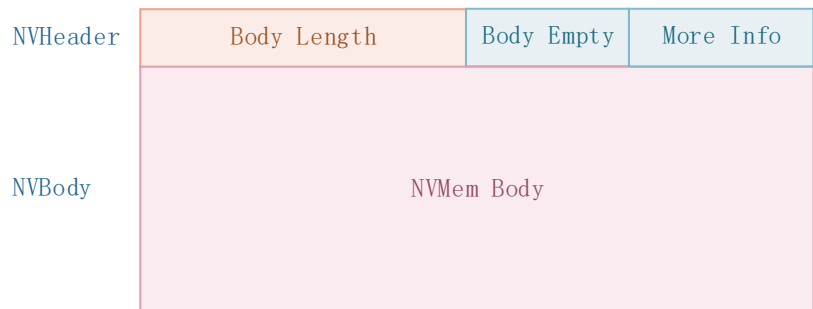


图 8：低功耗处理开机流程图

流程说明及建议：

1. 低功耗应用的上下文恢复建议在 `appInit` 函数中执行。每个应用对应一个低功耗入口函数 `xxxEngineInit()`，同时每个应用对应一个用于保存恢复用的上下文全局变量 `gXXXContext`，以及一个 NVM 数据文件 `xxx_config.nvm`。NVM 数据文件的格式可以是以下形式（仅供参考），NVHeader 包含 Body Length，Body Empty Flag 以及其他标记信息



Body Length：标记 NV 数据区实际的长度，便于按长度读取出 NVMem Body

Body Empty：标记 NVMem Body 区域是否写过数据了，假如无数据，可以不用做恢复或擦除操作

More Info: 用户可以根据自己的需求补充一些其他信息用于确保文件的完整性，譬如 MagicWord, CheckSum, Version 等

2. 用户需要自行保证 NVM 数据文件的完整性，如果检测到文件不完整，需要恢复默认值。可能有如下情况：
 - (1) 第一次烧写版本或 NVM 数据擦除了，flash 为空。不存在该文件，需要初始化并做一次写入。
 - (2) 程序升级后文件长度更改或存储区发生偏移时需要考虑有恢复机制，可考虑使用 checksum 等方式检查出文件异常并擦除文件重新初始化。
3. 使用 NVHeader 的目的是可以减少大片读取 Flash 的可能，仅当判断数据区有数据才去做读取。另外可帮助实现 NVMem Body 的完整性检查。
4. 做完完整性检查后，查询从哪个睡眠深度唤醒的。假如是 Power ON 上电后运行到此，那么用户可根据自己需求决定是否要去对该文件做一次擦除。假如不擦除并恢复出来，那么这次上电后会产生类似从睡眠唤醒的效果，这些用户可以根据应用需求决定。
5. 假如从 Slp2/Hib 状态唤醒，先判断一下 Body Empty 标志，假如后续 NVMem 有效，那么读出并恢复到 ram。
6. 上下文恢复后，用户应创建自己的 APP 任务，随后在 APP 任务中去执行自己的流程
7. appInit 是在一个很高优先级的任务中被调用的，被用来先于整个系统做全局初始化。所以应用的上下文初始化时间需要尽可能的短，耗时的操作如等待网络连接，等待消息等操作应放到自己的应用任务中去处理。
8. 在我们的 AT Command 工程中，我们先恢复了上下文，再根据上下文信息去决定是否重启应用任务。假如应用任务一定会运行起来，那么上下文的恢复也可直接放到应用任务中去执行。总体的原则是不要在 appInit 中 block 太久。

4.1.2 应用上下文恢复相关 API

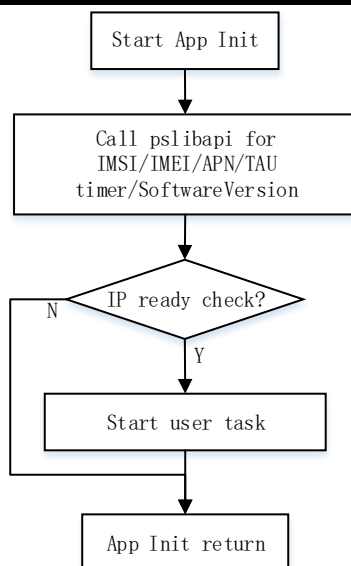
应用上下文信息我们建议存储在文件系统中，使用方便且有擦写均衡算法。文件系统的写入使用以下函数完成

```
OSAFILE OsaFopen(const char *fileName, const char* mode);
INT32 OsaFclose(OSAFILE fp);
UINT32 OsaFread(void *buf, UINT32 size, UINT32 count, OSAFILE fp);
UINT32 OsaFwrite(void *buf, UINT32 size, UINT32 count, OSAFILE fp);
INT32 OsaFseek(OSAFILE fp, INT32 offset, UINT8 seekType);
INT32 OsaFsize(OSAFILE fp);
UINT32 OsaFremove(const char *fileName);
```

4.1.3 应用低功耗恢复流程

假如用户的应用是与网络相关的，那么需要唤醒后等待网络准备好，再发起数据业务

电信 OceanConnect 低功耗恢复流程：



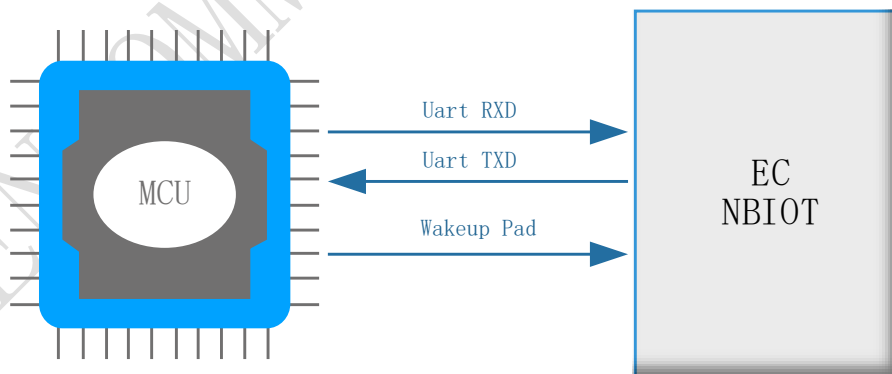
5. 双芯片方案的唤醒

当用户程序运行在 EC616 上，用户的程序可以通过 API 函数灵活的控制芯片的唤醒或睡眠。当 EC616 作为一个模块工作时，同样的将工作在低功耗模式下，外部 MCU 与 EC616 通信时需要遵循一个唤醒协议，唤醒 EC616 后才可发送数据。我们提供两种唤醒方案，每种方案的特点各不相同：

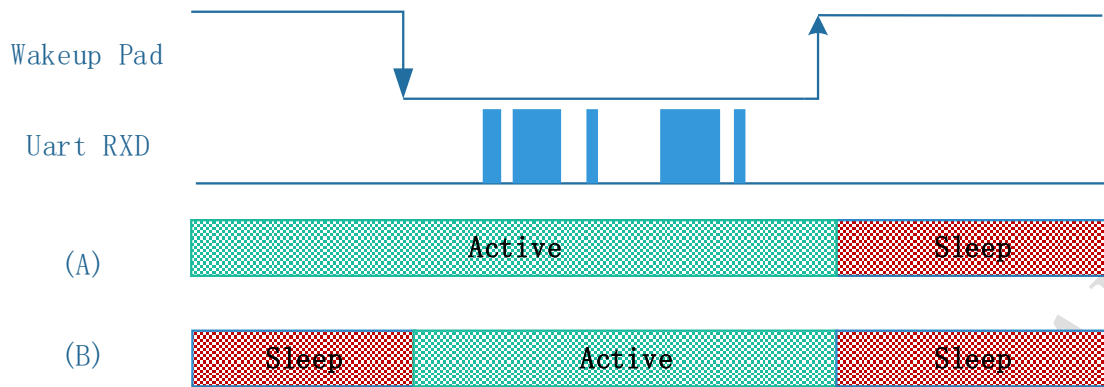
对于 EC616s，与 EC616 最大不同点在于拥有低功耗 UART，因此在双芯片方案中无需考虑芯片的唤醒，在任何时候直接发送 UART 数据即可。因此如下方案仅适用于 EC616。

方案一：

NB 和 MCU 通过 Uart 相连，并将 MCU 的一个 GPIO 接到 NB 的 Wakeup Pad 上，具体哪个 Pad 要与 NB 中运行的代码对应。这种方案下功耗最低，一旦命令收发完毕即可进入睡眠。

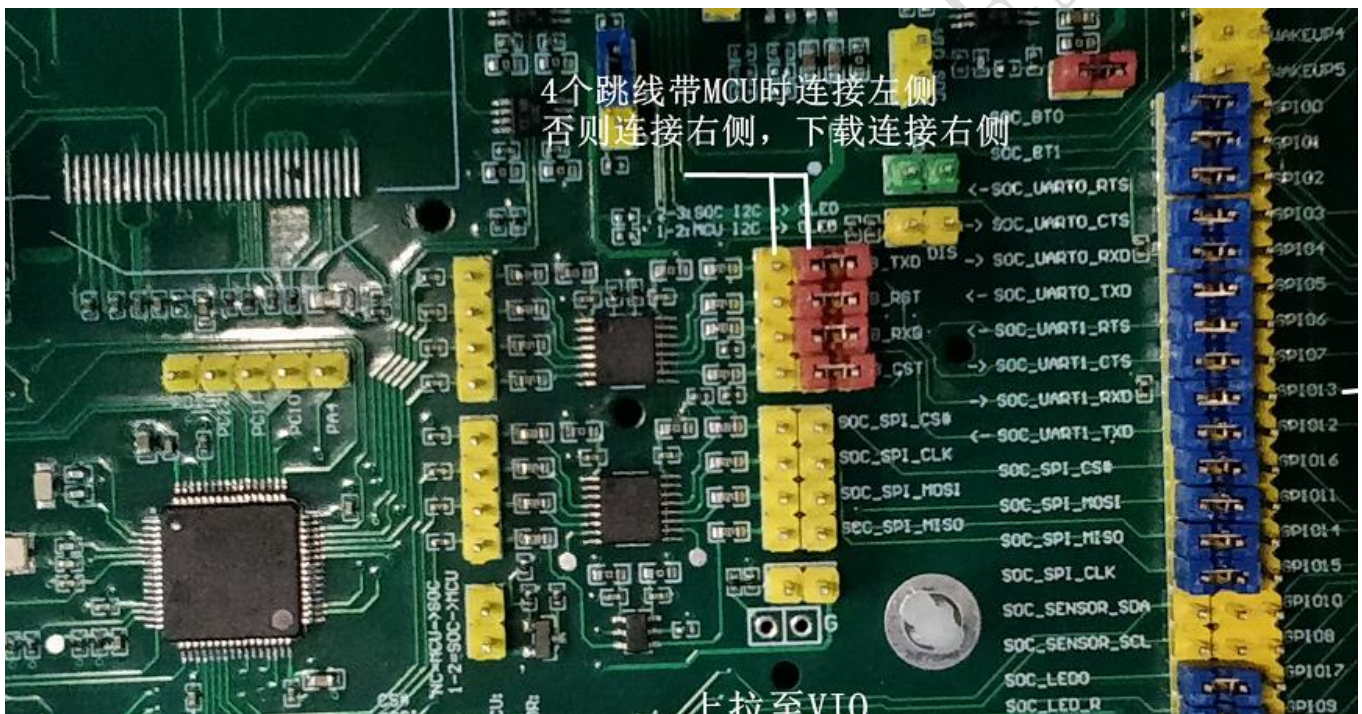


当 MCU 需要发送数据时，先将 Wakeup Pad 拉低。随后间歇地发送“AT\r\n”判断 NB 是否准备好，收到 OK 回复后，开始发送需要发送的 UART 指令，发送完指令后假如 MCU 并没有后续指令要发即可拉高 Wakeup Pad(不需要等到回复后再拉高)。情况 A：在 Active 状态（非睡眠状态）下，当 Wakeup Pad 为低时 NB 不会进入睡眠，此时可以顺利接收来自 MCU 的指令。情况 B：在 Sleep 状态下，当 Wakeup Pad 拉低 NB 唤醒，串口收发指令后拉高 Wakeup Pad，NB 进入睡眠。

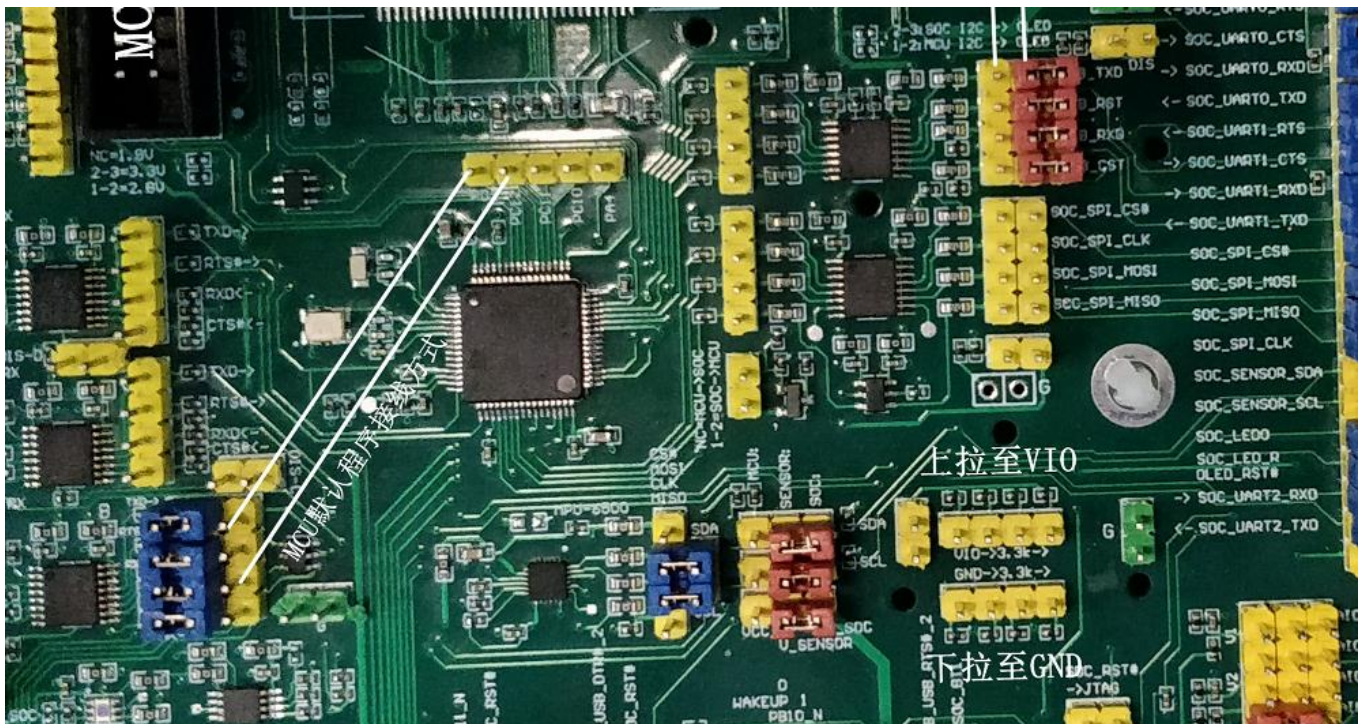


实验板跳线连接

使用方案一时，将下图 4 个跳线接到左侧。



另外，连接如下两根白线，即将 MCU 串口连接通底板外部接口。



MCU 串口位于底板左下侧。上侧的串口为供电接口，一拖四，NB 的 log 为其中第一个串口。



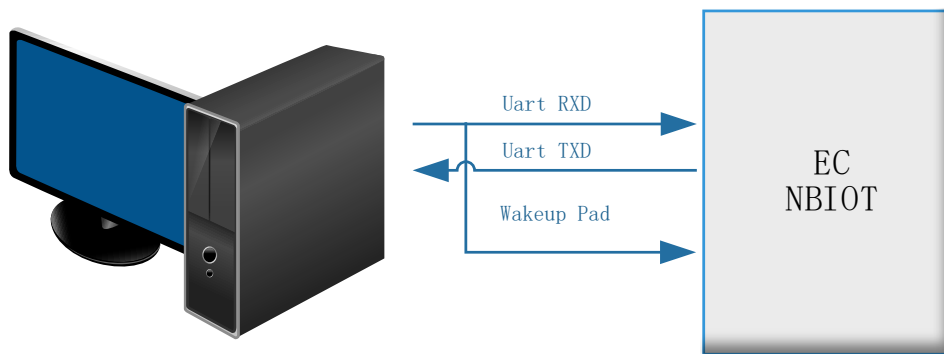
方案二：

NB 因外部中断唤醒系统后，会延迟一段时间再进入睡眠。这种方式下用户可以设置延时的时间，比较适合在不带 MCU 的情况下使用。可以通过 AT 命令设置唤醒后的延时时间，需要注意为保证设置成功需要在系统一上电时立即设置，因为此时 NB 还不会立即进入睡眠。设置后此配置立即写入 flash，在重新上电后生效。对应 AT 命令如下：

```
AT+ECPCFG="slpWaitTime", 4000
```

其中最后的 4000 代表唤醒后等待 4000ms 再进入睡眠，可根据需求设置任意值，最大范围是 65535ms。需要注意的是，每次唤醒的第一条命令会丢失，需要在延期内再次发送。此配置将写入 flash，休眠或硬件复位都不会清除该配置。

这种方案下可以采用如下接线方式：



NB 由 RXD 上的跳变电平唤醒，唤醒后启动延时定时器，并等待预期的时间后才使能 NB 的睡眠。此方案相比方案一更加简单直接，但由于采用固定的延时，在 UART 数据收发结束后会存在等待睡眠的空闲时间。另外，考虑到用户可能需要在收发一条 AT 指令后再次收发几条命令，那么可在收到 AT 指令后调用 `slpManStartWaitATTimer()` 再次延时一个 `slpWaitTimer` 时间。详见 SDK 中的 `at_command` 工程的 `app.c`。在 `atCmdProcessEntry` 中每收到一条消息都会调用 `slpManStartWaitATTimer()` 启动一个定时器，将睡眠时间延后 `slpWaitTimer` 毫秒。

```
static void atCmdProcessEntry(void *arg)
{
    ....uint32_t data_block_len;
    ....uint32_t at_last_cnt = 0;

    ....osMessageQueueAttr_t queue_attr;

    ....memset(&queue_attr, 0, sizeof(queue_attr));

    ....queue_attr.mq_size = sizeof(atcmd_queue_buf);
    ....queue_attr.mq_mem = atcmd_queue_buf;
    ....queue_attr.cb_mem = &atcmd_queue_cb;
    ....queue_attr.cb_size = sizeof(atcmd_queue_cb);

    ....// .init message queue
    ....mid_atcmd_msgqueue = osMessageQueueNew(MSGQUEUE_OBJECTS, sizeof(msgqueue_obj_t), &queue_attr);

    ....EC_ASSERT(mid_atcmd_msgqueue, mid_atcmd_msgqueue, 0, 0);

    ....// .wait for at command
    ....if(UsartAtCmdHandle != NULL)
    ....    UsartAtCmdHandle->Receive(at_uart_recv_buf, AT_CMD_BUFFER_SIZE);

    ....while(1)
    ....{
        ....msgqueue_obj_t msg;
        ....osStatus_t status;
        ....ARM_USART_STATUS_uart_status;
        ....uint32_t mask;

        ....// .wait for message
        ....status = osMessageQueueGet(mid_atcmd_msgqueue, &msg, 0, osWaitForever);

        ....slpManStartWaitATTimer(); ....// .Wait AT for slpWaitTime in order to receive next command

        ....if(status == osOK)
        ....{
            ....EC_ASSERT(msg.recv_cnt > at_last_cnt, msg.recv_cnt, at_last_cnt, 0);

            ....data_block_len = msg.recv_cnt - at_last_cnt;
```

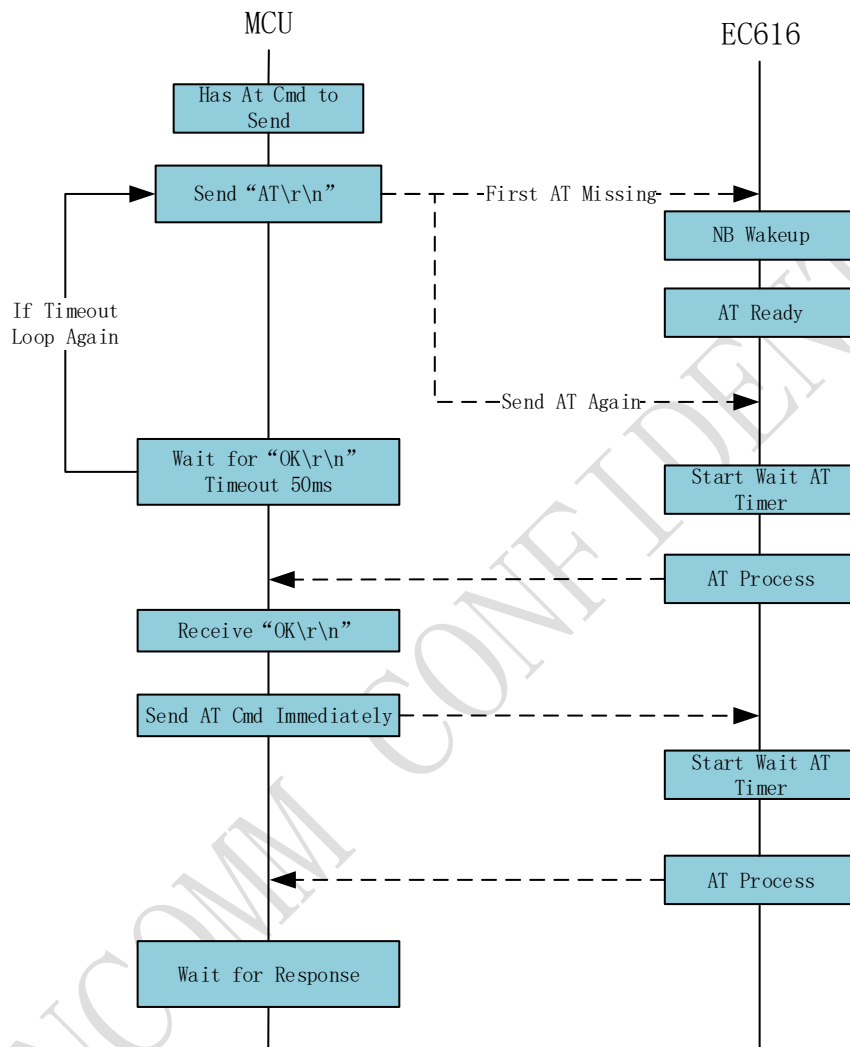
默认情况下，采用方案二会在两种情况下启动这个睡眠等待定时器：

- (1) 系统刚上电或刚唤醒
- (2) AT 串口收到一条命令（无论是有效指令，还是无效指令）

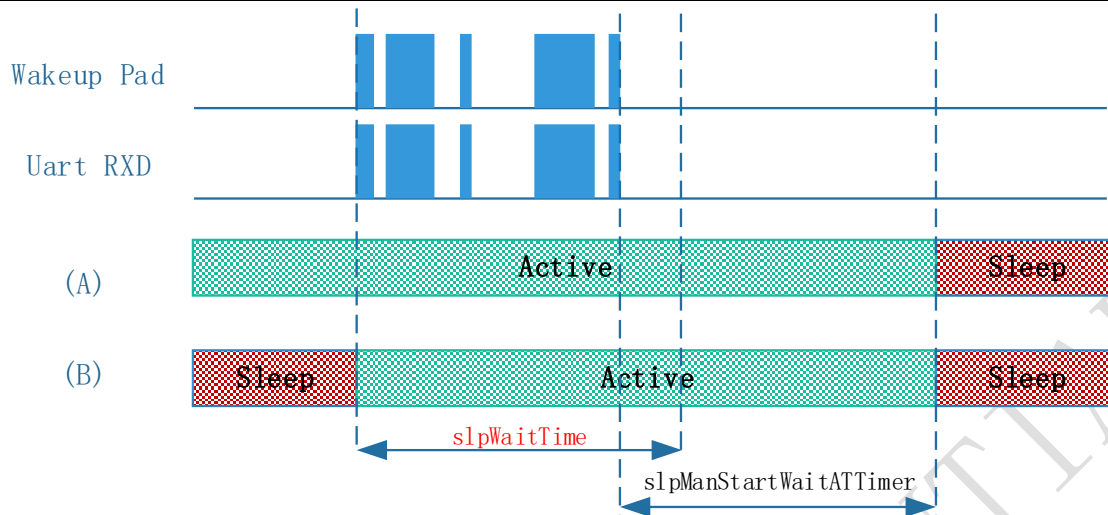
MCU 侧对于 EC616 的唤醒流程如下：

- (1) 无论 EC616 处于唤醒还是睡眠，MCU 在试图发送 AT 指令前都应该尝试先与 EC616 握手
- (2) MCU 发送 “AT\r\n” 作为握手信号，并等待 EC616 的回复。收到 “OK\r\n” 代表握手成功
- (3) MCU 每 50ms 发送一次握手信号，直到收到 EC616 的回复

- (4) 握手成功后，MCU 立即发送实际需要发送的 AT 指令，并等待回复（此时的 timeout 为对应 AT 指令的 Timeout）
- (5) 下图中 Start Wait AT Timer 执行后将等待 slpWaitTimer 毫秒，后一条 AT 指令需要在这个时间间隔内发送
- (6) 在 slpWaitTimer 后 EC616 可能会进入睡眠，但是否进入睡眠要综合考虑其他情况（其他投票结果）。

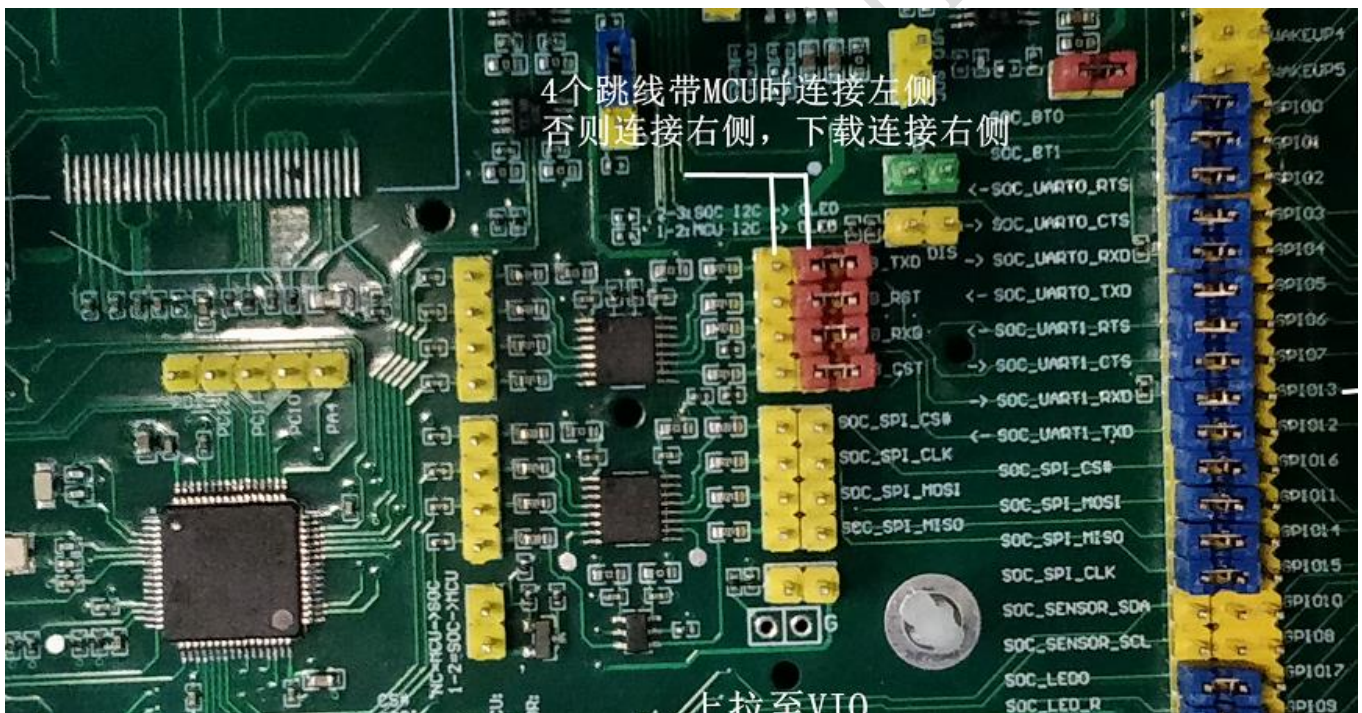


atCmdProcessEntry 每收到一条 AT 消息便会再次将睡眠时间再次向后延迟 slpWaitTime，例如原先睡眠延迟还剩余 2000ms 的情况下再次收到 AT 消息，睡眠延迟将会重新变为 slpWaitTime。



实验板跳线连接

下图四个跳线接到右侧，并连接 Wakeup1 和 GPIO13。配置 AT+EPCFG="slpWaitTime", 4000\r\n 后每次唤醒会保持 4000ms 不睡眠。为兼容模块版本，可直接将 Wakeup1 和 Wakeup3 连接在一起。





6. 测试

6.1 唤醒中断响应测试

EC616 从不同睡眠深度中唤醒的中断响应时间，测试过程中 NB 进入 PSM 模式：

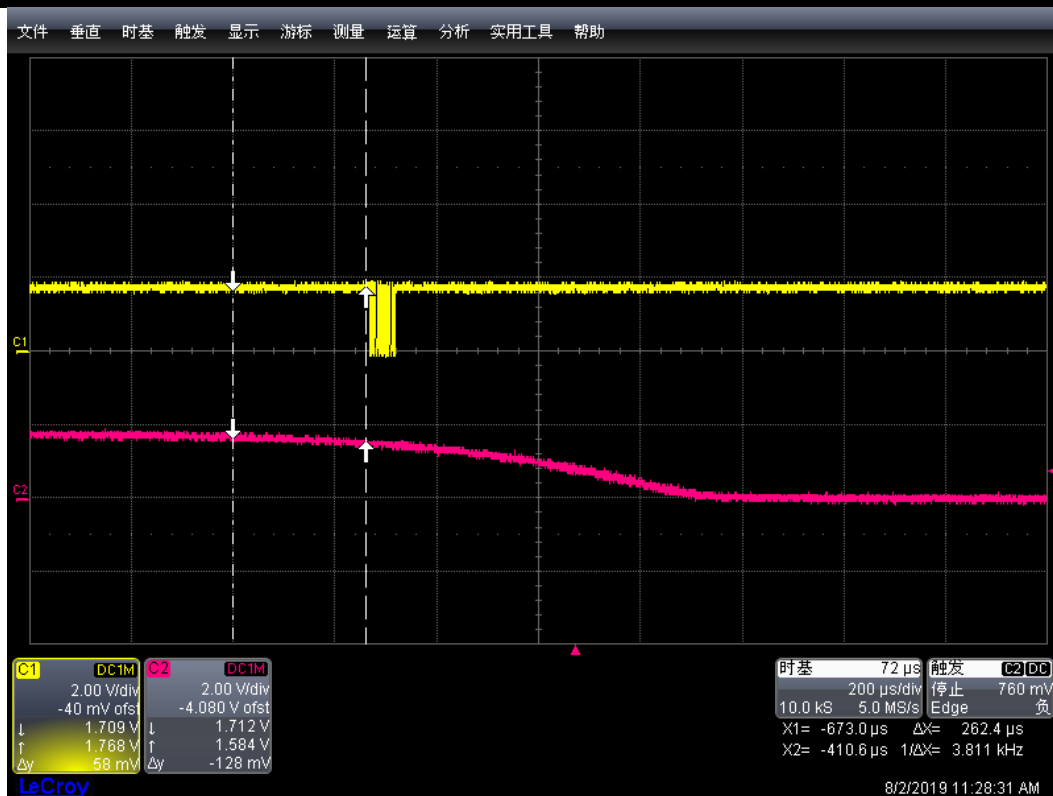
测试方法：

进入睡眠后，按 Wakeup0 唤醒，在 Pad0_WakeupIntHandler 中初始化 GPIO 并翻转 IO 电平多次，测量从按键触发到系统唤醒并执行中断处理函数的时间。ARM 启动后默认不使能中断，用户需要在 BSP_CustomInit() 中尽快使用 NVIC_EnableIRQ(PadWakeup0_IRQn) 启动 Wakeup0 中断。

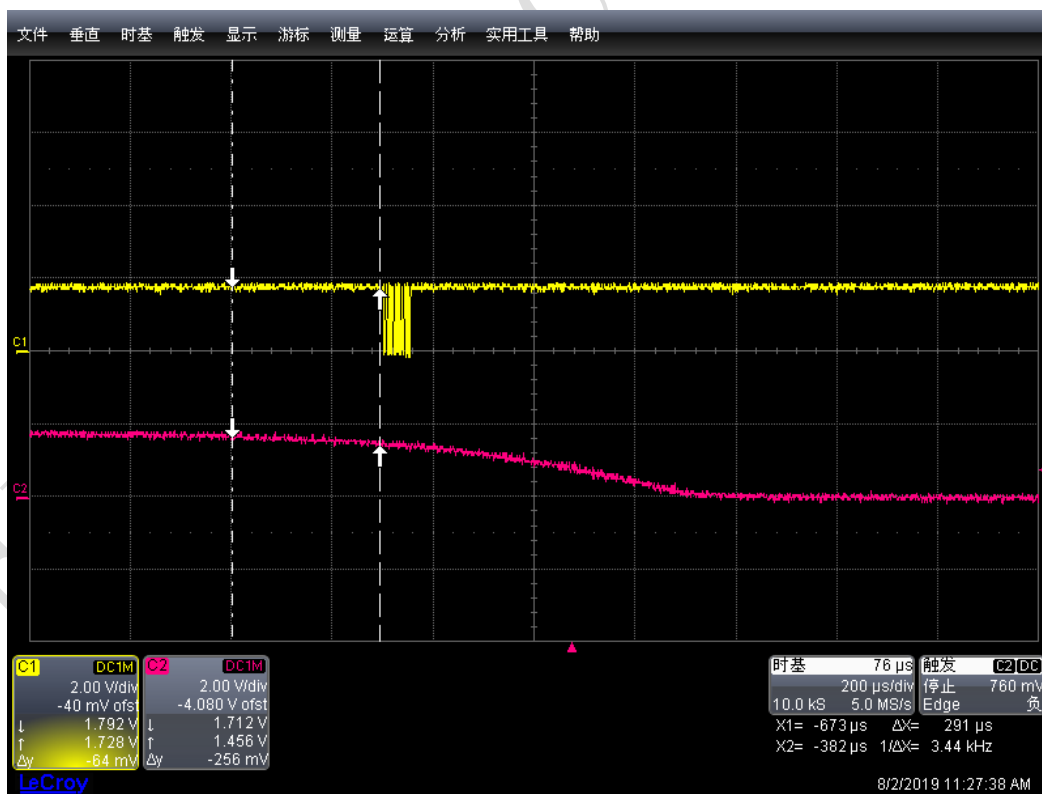
中断服务函数如下（用户可实现自己的服务函数）：

```
void Pad0_WakeupIntHandler(void)
{
    if(slpManExtIntPreProcess(PadWakeup0_IRQn)==false)
        return;
    //add custom code below//
    GPIO_Test_Init()
    for(uint8_t i;i<9;i++)
    {
        GPIO_Test_High();
        GPIO_Test_Low();
    }
}
```

Active 状态（不启用睡眠）情况下中断响应时间约 262us：

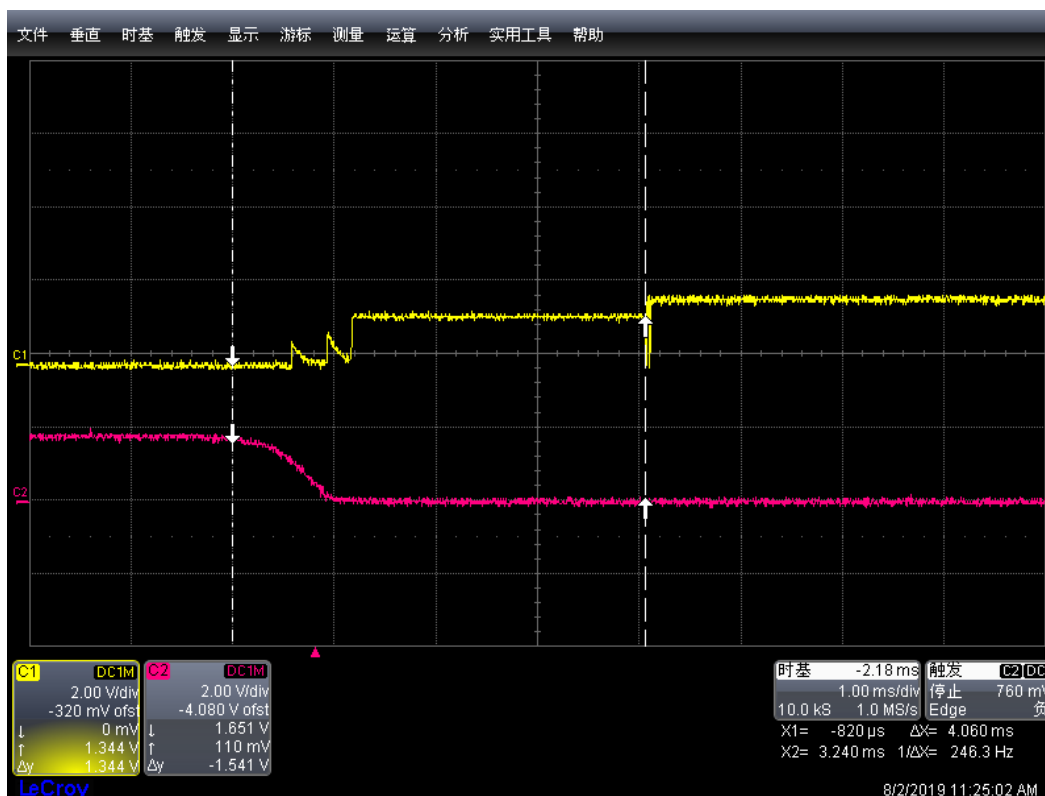


最大睡眠深度为 Idle 状态下中断响应时间约 290us:

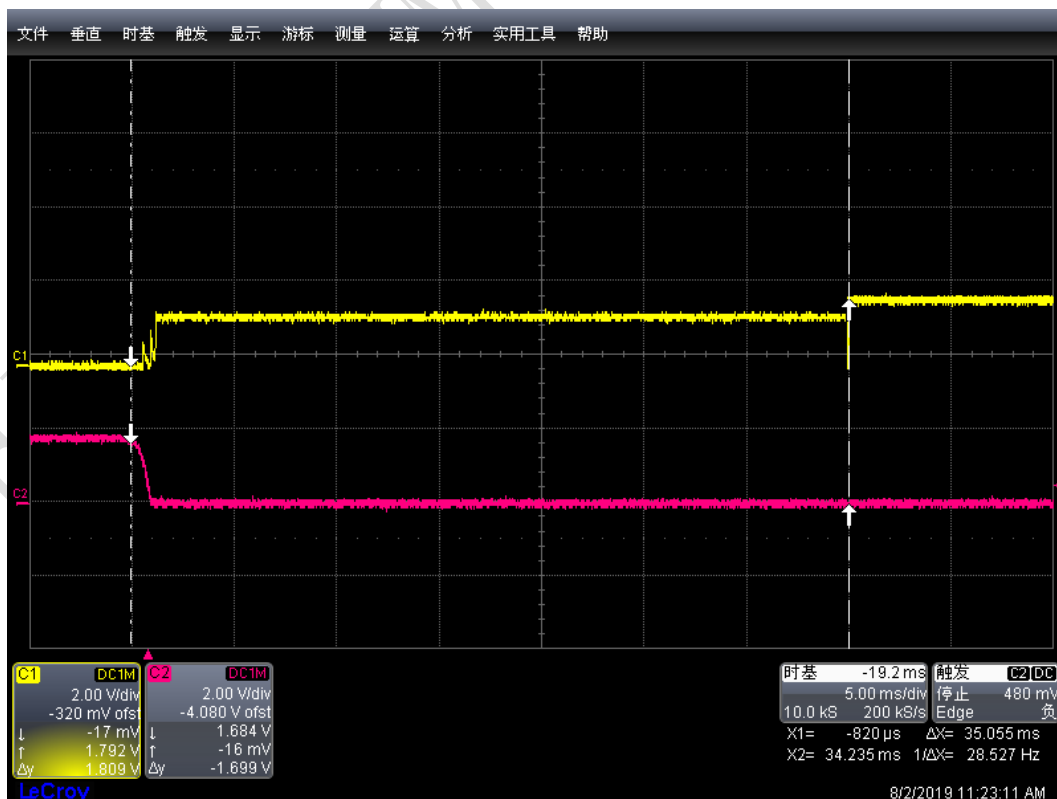


最大睡眠深度为 Sleep1 状态下中断响应时间约 4.1ms:

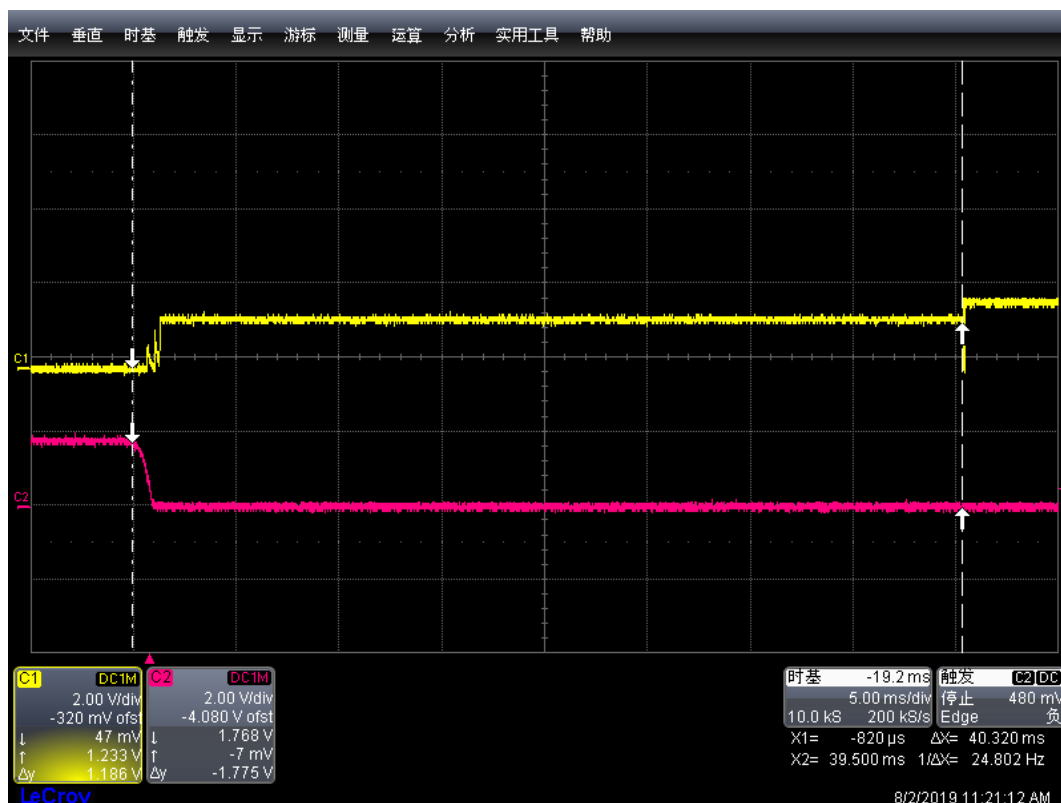
此处看见在中断执行前，IO 电平跳变及并未拉足到高的现象是正常的，因为上电后默认 IO 处于输入弱上拉状态，进入中断服务函数后，IO 被设置为输出态，才有了稳定的电平。后面从 Hibernate, Sleep2 的唤醒也是一样的现象。



最大睡眠深度为 Sleep2 状态下中断响应时间约 35ms:



最大睡眠深度为 hibernate 状态下中断响应时间约 41ms:



6.2 双芯片唤醒方案二测试

为了更加清晰地解释方案二的唤醒方式，在 PC 上运行 python 脚本来演示这个唤醒过程。通过 API 的封装可以实现对 AT 发送进程无感。

测试方法：确保 UART1_RXD 连接到了 Wakeup0（默认 wakeup0 与底板按键连接，需要拔掉那个跳线帽），底板连接 PC 并，找到对应的 UART 接口。本例子中 AT UART 接口映射为 COM12。确保配置了 AT+ECPCFG="slpWaitTime",4000 此处 4000 可根据测试情况调小，自动化的测试流程中通常 1000ms 以内即可完成。

- (1) Ec616_UartSend(at_str) 此函数为最终封装后给应用层发送 AT 的 API
- (2) 在 AT Command 正式发送之前，会先有一个握手流程，用来唤醒 EC616。见 Ec616_Uarthandshake()。
在 Ec616_Uarthandshake()每间隔一段时间发送一次 AT 并等待 OK 的回复，一旦收到 OK 回复，即代表握手成功。
- (3) 握手成功后立即发送实际需要发送的 AT 指令，此时用户可根据不同的 AT 指令，设置不同的 Timeout 等待时间
- (4) 在 Timeout 时间内等待 AT 的回复。
- (5) Ec616_Uarthandshake()可通过封装隐藏其实现。Polling_OK()是等待接收 OK 字符的过程，MCU 端可以有其他的实现方式。MCU 侧在发送 AT 指令时，真正调用的是 Ec616_UartSend(at_str)。

```

2  import serial
3
4  ser = serial.Serial('COM12',115200)
5
6  AT=b'AT\r\n'
7  CESQ=b'AT+CESQ\r\n'
8
9
10 def Polling_OK():
11     for i in range(1, 10):
12         data = ser.readline()
13         if(data == b'OK\r\n'):
14             print(data)
15             return True;
16         elif(data != b''):
17             print(data)
18     return False;
19
20 def Ec616_Uarthandshake():
21     ser.timeout = 0.1
22     print("EC616 handshake Start");
23     for i in range(1, 10):
24         ser.write(AT)
25         if(Polling_OK() == True):
26             print("EC616 Wakeup Complete");
27             return True
28     return False;
29
30 def Ec616_UartSend(at_str):
31
32     if(Ec616_Uarthandshake()):
33         ser.timeout = 10
34         ser.write(at_str)
35
36         Polling_OK();
37     else:
38         print('Wakeup Timeout')
39
40
41 def WakeupTest_Start():
42     print('Start Test')
43     if(ser.is_open == True):
44         ser.close()
45     ser.open()
46     Ec616_UartSend(CESQ)
47     ser.close()
48
49
50 if __name__ == '__main__':
51     WakeupTest_Start()
52
53

```


7. 错误码

供用户调用的所有 PMU 相关函数都会提供返回值，建议调用函数后检查返回值，确保返回值为 RET_TRUE。其他情况均说明函数执行存在问题。相应返回值说明如下：

RET_TRUE,	/**< Error code: no error */
RET_UNKNOW_FALSE,	/**< Error code: unknow error */
RET_CB_ARRAY_FULL,	/**< Error code: callback array is full */
RET_CALLBACK_EXIST,	/**< Error code: duplicate callback in callback array */
RET_CALLBACK_UNEXIST,	/**< Error code: callback unexist */
RET_VOTE_SLP_OVERFLOW,	/**< Error code: vote overflow (above 256) */
RET_VOTE_SLP_UNDERFLOW,	/**< Error code: vote underflow (below 0) */
RET_ZERO_NAME_LEN,	/**< Error code: input name is empty */
RET_INVALID_NAME,	/**< Error code: input name has illegal character */
RET_VOTE_HANDLE_FULL,	/**< Error code: no place for more vote handle */
RET_INVALID_HANDLE,	/**< Error code: vote handle is invalid */
RET_INVALID_VOTE,	/**< Error code: the vote state is invalid */
RET_VOTE_CONFLICT,	/**< Error code: vote conflict */
RET_GIVEBACK_FAILED,	/**< Error code: vote handle give back error */
RET_HANDLE_NOT_FOUND,	/**< Error code: vote handle not found */

8. 注意事项及常见问题

8.1 API 使用限制及解决方法

EC616/616s 作为一款 NB MCU，在执行用户代码的同时还要确保 NB 协议栈的正确运行，因此在使用中有一些限制。在此仅描述低功耗相关的限制。

(1) OSDelay 的计数只有在 Idle 及 Sleep1 流程中有效

由于 Sleep2 及 Hibernate 状态下 sram 会大部分掉电或全部掉电，因此运行在其上的 RTOS 系统也无法继续维持运行。即唤醒后，RTOS 将重新开始运行，因此 OSDelay 信息将丢失。

用户如果想维持 OSDelay 有效，则不应该投票进入 Sleep2 及 Hibernate。假如投票了，则认为所有 OSDelay 的延时可以不再考虑。

如何开启一个各状态下有效的 Timer：

DeepSleep Timer 在 Idle, Sleep1, Sleep2, Hibernate 下均能够维持。

(2) DeepSleep Timer 仅有 8 个，其中 ID0-6 计数范围最大 580 小时，ID7 计数范围最大 2 小时，且精度为 1s

用户可以指定不同的 Timer ID，但同时存在的 Timer 个数仅能为 8 个。建议尽可能的少用 DeepSleep Timer，尤其是防止启用过短的 DeepSleep Timer，因为它会限制连续睡眠，且每次重建 timer 或超时都会存在一次 flash 写入（ID7 不会写入到 flash）。频繁的进出睡眠不利于降低功耗。

(3) 睡眠前后的回调函数

睡眠前后的回调函数在睡眠前一刻被调用，此时系统关闭了中断，因此同样不存在任务调度。因此，回调函数不能使用收发消息，挂起任务等操作。由于所有回调函数的运行是在关中断的状态下，因此所有用户注册的回调函数执行时间不得大于 3ms。请在执行完耗时操作后才投票进入睡眠。

- (4) 如用户的应用流程中启用了 ADC 采集功能，那么在完成应用流程并投票睡眠前需要关闭对应的 ADC 通道，即调用 ADC_ChannelDeInit。仅当使用过的 ADC 通道都被关闭后，睡眠电流才能稳定在最优状态。若应用流程中，并无使用 ADC 的情况（即并没有调用过 ADC_ChannelInit），那么无需特意关闭 ADC。

9. 版本

版本	日期	备注
Draft	2019-03-20	wqzhao: 基础版本
V1.1	2019-10-21	wqzhao: 加入 slpManGetWakeupSrc 的说明
V1.2	2019-11-28	wqzhao: 加入如何理解投票函数名称的说明
V1.3	2020-03-25	wqzhao: 应用低功耗实现章节描述了一个更通用的流程
V1.4	2020-04-06	wqzhao: 加入 EC616s 的相关说明
V1.5	2020-05-11	wqzhao: AONIO 使用说明
V1.6	2020-05-20	wqzhao: DEEPSLP_TIMER_ID7 特殊说明

10. 关于我们

上海移芯通信科技有限公司坐落于上海张江，公司于 2017 年 2 月成立，从事蜂窝移动通信芯片的研发和销售。公司产品主要应用于广域物联网通信领域，致力于设计最具性价比的蜂窝物联网芯片。公司团队在蜂窝通信芯片上有着辉煌历史和丰富经验。公司所有核心技术全部自研，包括算法&架构、射频、基带、SoC、协议栈软件、平台&应用软件和硬件方案等。

移芯通信首款自主研发的超低功耗 NB-IoT 芯片 EC616 于 2019 年中量产，被绝大部分头部模组企业采用，现已大批量应用于全国各区域各行业。下一代超高集成度 NB-IoT 芯片 EC616S 于 20 年底量产，超低功耗 4G Cat.1 芯片 EC618 工程样片阶段，5G 芯片正在研发中。

上海移芯通信科技有限公司

地址：中国上海市浦东新区祥科路 298 号佑越国际 1 幢 7 层

邮编：201210

电话：

电邮：

网址：<http://www.eigencomm.com>

技术支持窗口

电邮：support@eigencomm.com